



# Instituto Politécnico Nacional

## Escuela Superior de Computo



**Materia:** Computación Grafica

**Profesor:** Rosaura Palma Orozco

**Alumno:** Gil Valentín Oswaldo

**Grupo:** 6CV2      **Fecha:** 02/06/2026

### Instrucciones:

El documento deberá describir detalladamente el desarrollo del proyecto, desde el planteamiento del problema hasta su implementación, haciendo énfasis en las técnicas de graficación por computadora utilizadas y la justificación de las herramientas empleadas.



# 1. Introducción

## Contexto general

En el ámbito de la manufactura moderna, los procesos de corte de materiales mediante tecnologías CNC (Control Numérico Computarizado), tales como el corte por láser, chorro de agua o plasma, demandan una precisión milimétrica. No obstante, más allá de la precisión del maquinado, la eficiencia económica de una planta de producción está directamente ligada al máximo aprovechamiento de la materia prima (láminas de acero, madera, textiles o polímeros). La distribución óptima de piezas de formas irregulares sobre una superficie bidimensional delimitada se conoce formalmente como el *Problema de Anidamiento* o *Nesting*.

## Importancia de la graficación en el problema

El problema del *Nesting* es de naturaleza puramente geométrica y espacial. Tradicionalmente, la manipulación de formas complejas requiere de representaciones matemáticas abstractas que resultan difíciles de validar sin una retroalimentación visual directa. La computación gráfica actúa como el puente crítico entre los algoritmos analíticos de optimización y el operador humano. A través de la discretización de formas en mallas poligonales, la aplicación de transformaciones matriciales en coordenadas homogéneas y la renderización en tiempo real de geometrías afines, la graficación permite interactuar, calcular y previsualizar con total exactitud matemática el comportamiento y la colisión espacial de los materiales antes de ejecutar un corte físico irreversible.

## Descripción breve del proyecto

Este proyecto consiste en el desarrollo de un sistema de software estático de diseño geométrico asistido enfocado en la optimización del aprovechamiento de material. El sistema procesa un conjunto de vértices dispersos que definen piezas irregulares en un espacio euclídeo, calcula de forma automatizada su Envolvente Convexa (Convex Hull) mediante el algoritmo matemático de *Graham Scan*, y permite al usuario realizar transformaciones geométricas rígidas (traslaciones y rotaciones matriciales) sobre una lámina de dimensiones normalizadas. Finalmente, si una geometría excede las fronteras físicas de la lámina de trabajo, el sistema ejecuta un algoritmo de Recorte de Polígonos (Sutherland-Hodgman) para recortar los excedentes y calcular analíticamente los coeficientes de área útil frente al desperdicio de material.

## 2. Planteamiento del problema

### Descripción detallada del problema

Cuando se fabrican piezas con morfologías irregulares (cóncavas, con perforaciones o bordes no ortogonales), calcular visual o manualmente la disposición ideal de los patrones sobre una plancha de material genera pérdidas geométricas masivas. Si el software de corte agrupa las piezas utilizando contenedores rectangulares estándar (*Bounding Boxes* ortogonales), las concavidades e irregularidades intrínsecas de cada pieza quedan aisladas, impidiendo que otras piezas ocupen esos espacios vacíos. Esto se traduce en un alto índice de desecho de material por cada ciclo de producción. El núcleo del problema radica en procesar computacionalmente la convexidad de las figuras para aproximar de manera óptima las distancias mínimas de aproximación entre perímetros poligonales individuales.

### Situación actual

En talleres de manufactura mediana o entornos educativos de prototipado, el acomodo de piezas suele realizarse "al tanteo" mediante software de dibujo vectorial genérico, o bien mediante el uso de suites CAM comerciales cuyas licencias son sumamente costosas y ocultan el pipeline matemático detrás de cajas negras de software cerrado. Esto limita la capacidad de los ingenieros para auditar las tolerancias analíticas de los algoritmos de recorte de polígonos (*clipping*) o para personalizar restricciones de convexidad específicas en geometrías complejas.

### Justificación

El desarrollo de esta herramienta computacional se justifica por la necesidad de proveer una solución matemática transparente y de bajo costo computacional orientada a la optimización de recursos en ingeniería. Desde la perspectiva académica de la Computación Gráfica, el proyecto integra de manera orgánica los conceptos de subespacios afines, matrices de transformación homogénea, algoritmos de generación de primitivas bidimensionales y técnicas de clipping lineal/poligonal, materializando la teoría matemática abstracta en una herramienta de optimización industrial tangible y auditable.

### Alcances y limitaciones

- **Alcances:** El sistema procesará geometrías arbitrarias definidas por vértices ingresados por el usuario. Realizará cálculos de convexidad en tiempo real, transformaciones matriciales completas sobre los polígonos, detección visual de colisiones elementales por solapamiento mediante código de colores, y

recorte exacto de polígonos contra ventanas de visualización/corte rectangulares.

- **Limitaciones:** En estricto cumplimiento de las restricciones metodológicas de la asignatura, el sistema **no implementará simulaciones físicas dinámicas ni animaciones fluidas basados en variables de tiempo**. Los cálculos de posicionamiento son discretos y estáticos. El procesamiento geométrico se limitará al espacio bidimensional ( $\mathbb{R}^2$ ), absteniéndose de proyectar volúmenes en perspectivas tridimensionales complejas de la Unidad 2.

### 3. Objetivos

#### 3.1 Objetivo general

Diseñar e implementar un sistema de software geométrico para la optimización y anidamiento (*Nesting*) de piezas irregulares sobre láminas de trabajo físicas, mediante la aplicación analítica de algoritmos de Envolvente Convexa, Transformaciones Afines Matriciales y Recorte Poligonal de Sutherland-Hodgman, prescindiendo del uso de simulaciones o animaciones temporales.

#### 3.2 Objetivos específicos

1. Implementar una estructura de datos basada en mallas y vectores para modelar puntos y subespacios afines euclídeos en  $\mathbb{R}^2$ .
2. Desarrollar un módulo matemático capaz de evaluar la convexidad de un conjunto de puntos dispersos y computar de manera eficiente su *Convex Hull* utilizando el algoritmo matemático de *Graham Scan*.
3. Construir un motor de transformaciones geométricas bidimensionales que aplique matrices de multiplicación en coordenadas homogéneas para realizar traslaciones y rotaciones precisas sobre las geometrías cargadas.
4. Codificar el algoritmo de recorte de polígonos de *Sutherland-Hodgman* para delimitar y seccionar analíticamente las geometrías que superen los bordes del área útil del material.
5. Desarrollar una interfaz gráfica estática de alta fidelidad que emplee modelos de color discretos (RGB) para representar zonas de corte válidas, colisiones espaciales y áreas de desperdicio.

## 4. Marco teórico

### Conceptos fundamentales de graficación por computadora

La computación gráfica descansa sobre cimientos del álgebra lineal y la geometría analítica. Un punto en el espacio gráfico se define como un vector de posición dentro de un Espacio Afín Euclídeo, el cual combina un espacio vectorial con un conjunto de puntos sobre los que se definen nociones métricas como distancia y ángulo.

La manipulación eficiente de estos puntos requiere el uso de Coordenadas Homogéneas. Este sistema añade una dimensión adicional a los vectores (un punto  $(x, y)$  se representa en un subespacio proyectivo como  $(x, y, 1)$ ), lo que permite unificar transformaciones no lineales como la traslación y transformaciones lineales como la rotación y el escalamiento bajo una misma operación matemática: la multiplicación de matrices.

### Técnicas utilizadas

- **Modelado Poligonal:** Representación de formas complejas por medio de mallas de polígonos (en este caso, polígonos cerrados definidos por listas indexadas de vértices conexos).
- **Convexidad y Envoltentes Convexas:** Un conjunto de puntos en el espacio es convexo si para cualquier par de puntos dentro del conjunto, el segmento de recta que los une está contenido por completo en él. La Envoltente Convexa (*Convex Hull*) de un conjunto de puntos  $S$  es el polígono convexo mínimo que encierra a todos los puntos de  $S$ .
- **Recorte de Polígonos (Polygon Clipping):** Proceso algorítmico que intercepta un polígono (sujeto) contra una región cerrada (ventana de recorte). A diferencia del recorte de líneas, el recorte de polígonos debe devolver nuevas geometrías cerradas, recalculando los vértices de intersección sobre los límites de la ventana.

### Tecnologías relacionadas

Para el desarrollo de herramientas CAD/CAM estáticas de alta precisión, se utilizan entornos de desarrollo gráfico declarativo de última generación. En este proyecto se empleará Kotlin como lenguaje de programación tipado, utilizando la librería nativa Jetpack Compose (Canvas API). Esta tecnología permite interactuar directamente con el búfer de pantalla y despachar operaciones matriciales vectoriales sin depender de motores físicos ni de videojuegos, garantizando un renderizado matemático puro.

## 5. Propuesta de solución

### Descripción general de la solución

Se plantea el desarrollo de una aplicación de escritorio o interfaz gráfica interactiva estática estructurada en tres capas lógicas:

1. **Capa Matemática de Geometría Computacional:** Encargada de almacenar los arreglos de vértices, computar las matrices de transformación homogénea de  $3 \times 3$ , procesar la envolvente convexa y ejecutar las funciones de clipping.
2. **Capa de Gestión de Escena y Estado:** Controla las coordenadas de la plancha de material (la cual actúa como la ventana de recorte o *Clip Window*) y las posiciones estáticas relativas de las piezas cargadas.
3. **Capa de Renderizado Gráfico:** Dibuja sobre un lienzo digital interactivo (*Canvas*) los polígonos resultantes, coloreando las superficies de acuerdo a criterios geométricos estrictos.

### Enfoque adoptado

El enfoque es puramente algorítmico y determinista. En lugar de procesar cuadros por segundo (FPS) en un ciclo de animación continua, la interfaz se redibuja de manera reactiva únicamente cuando ocurre un evento de entrada discreto (por ejemplo, cuando el usuario cambia numéricamente el ángulo de rotación o desplaza las coordenadas de una pieza). El cálculo del aprovechamiento de material se actualiza de manera analítica mediante la fórmula del área de polígonos de Gauss (*Shoelace formula*) aplicada a los polígonos recortados y válidos.

### Justificación técnica de la propuesta

El uso de coordenadas homogéneas garantiza que múltiples transformaciones sobre una pieza (por ejemplo, rotarla  $45^\circ$  y luego trasladarla 200 unidades en el eje X) puedan combinarse en una única matriz resultante mediante la composición por multiplicación matricial ( $M_{\text{total}} = T \times R$ ), optimizando el rendimiento. El uso del algoritmo de *Graham Scan* para la envolvente convexa proporciona una complejidad temporal óptima de  $O(n \log n)$  para el procesamiento de las piezas, mientras que el algoritmo de *Sutherland-Hodgman* es el estándar de la computación gráfica para clipping debido a su capacidad para procesar secuencialmente el polígono contra cada plano de corte de la ventana de forma predecible y exacta.

## 6. Desarrollo del proyecto

### 6.1 Análisis

#### Requerimientos funcionales

- **RF-01 Ingreso de Puntos:** El sistema debe permitir al usuario capturar puntos coordenados de manera numérica o mediante clics espaciales sobre el lienzo.
- **RF-02 Generación de Envolvente:** El sistema debe calcular y trazar automáticamente la envolvente convexa mínima que contenga los puntos ingresados.
- **RF-03 Transformaciones Afines:** El usuario debe poder ingresar parámetros discretos para trasladar ( $dx$ ,  $dy$ ) y rotar (ángulo) la pieza seleccionada sobre el espacio de trabajo.
- **RF-04 Recorte Automático:** El sistema debe evaluar instantáneamente si el polígono interseca los límites de la lámina de corte y recortar las secciones excedentes.
- **RF-05 Métricas de Aprovechamiento:** El sistema debe desplegar en texto el área total de la lámina, el área ocupada por las piezas y el porcentaje de desperdicio de material.

#### Requerimientos no funcionales

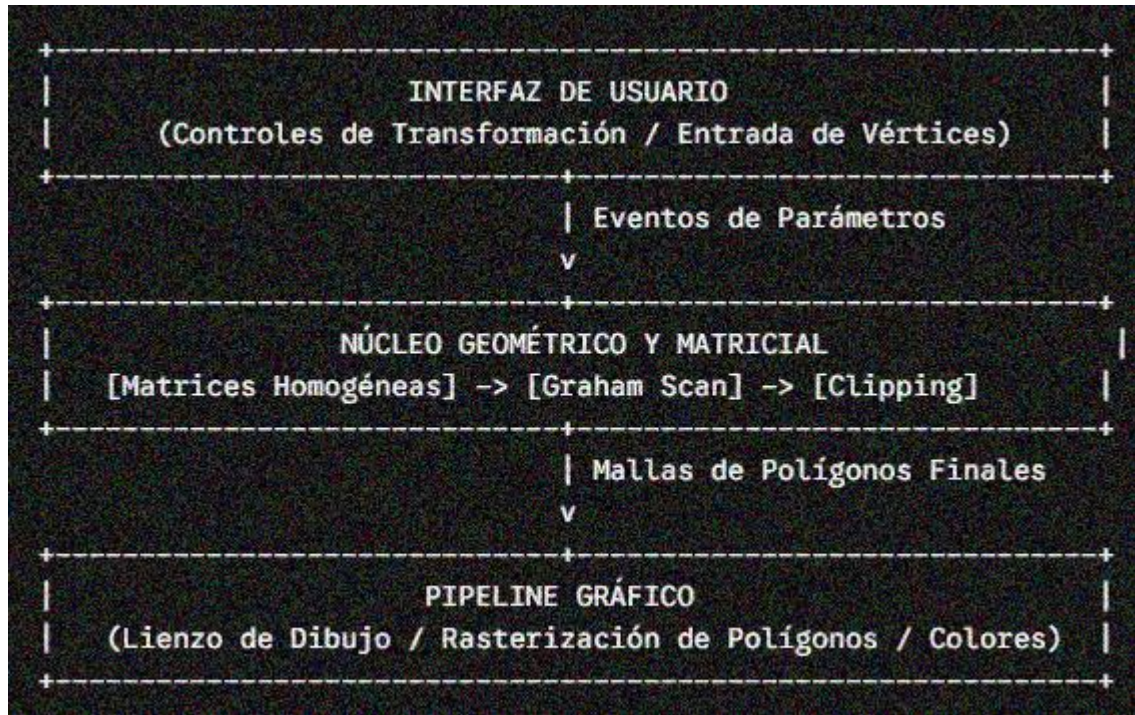
- **RNF-01 Precisión Numérica:** Los cálculos de coordenadas, intersecciones y áreas deben realizarse con precisión de punto flotante de doble precisión (double o float64).
- **RNF-02 Comportamiento Estático:** El sistema no debe contener hilos de ejecución dedicados a bucles de juego (*game loops*) ni refrescos temporizados automáticos. La renderización es gatillada por demanda de eventos.
- **RNF-03 Portabilidad/Disponibilidad:** El entorno gráfico debe implementarse bajo librerías estándar portables que no requieran hardware de aceleración 3D avanzado.



## 6.2 Diseño

### Arquitectura del sistema

El sistema se diseña bajo un patrón de arquitectura desacoplado donde la representación matemática abstracta es completamente independiente de la biblioteca gráfica utilizada para el dibujo en pantalla.



### Diagramas de flujo

El flujo lógico del procesamiento geométrico ante el cambio de estado de una pieza se describe secuencialmente en el siguiente bloque conceptual:

1. **Inicio:** El usuario modifica la posición o rotación de una pieza.
2. **Construcción de Matriz:** Se genera la matriz homogénea de transformación 3 X 3.
3. **Transformación Afín:** Se multiplican todos los vértices originales de la pieza por la matriz resultante.
4. **Cálculo de Convexidad:** Se ejecuta *Graham Scan* sobre los vértices transformados para determinar el área de su contenedor convexo.
5. **Evaluación de Límites:** ¿Los vértices transformados se encuentran fuera de la ventana de la lámina de corte?



- *Sí*: Se ejecuta el pipeline de *Sutherland-Hodgman* contra los 4 bordes de la lámina, generando un nuevo conjunto indexado de vértices.
  - *No*: Se conserva el polígono transformado original intacto.
6. **Cálculo Matemático de Áreas**: Se aplica el algoritmo de *Shoelace* sobre el polígono final (recortado o no) para actualizar los indicadores de rendimiento del material.
  7. **Rasterización y Despliegue**: Se envía el polígono final al pipeline de dibujo, aplicando coloración selectiva según su estado de colisión/recorte.

### Diseño de interfaces gráficas

La pantalla se subdivide en dos regiones principales fijas:

- **Panel de Control Lateral (Izquierdo)**: Contiene campos numéricos y deslizadores estáticos para configurar las coordenadas del punto de pivote, los desplazamientos en píxeles o milímetros (dx, dy), el ángulo de orientación en grados, y el botón para disparar el cálculo de optimización.
- **Lienzo de Trabajo Primario (Derecho)**: Un viewport rectangular que dibuja de forma estática una caja central que representa los límites físicos de la lámina de material (Ventana de Corte). Sobre este lienzo se proyectan las piezas.

## 6.3 Construcción (Implementación)

### Herramientas utilizadas

El desarrollo se estructura sobre el Python 3 / Matplotlib / Tkinter. Estas herramientas proveen control total sobre los vectores de primitivas y la manipulación de píxeles, permitiendo trazar envolventes convexas y realizar recortes analíticos de polígonos de forma totalmente estática.

### Algoritmos empleados

#### A. Representación Matricial y Coordenadas Homogéneas (Unidad 1.3.2)

Cualquier transformación sobre un vértice  $P=[x, y]^T$  se computa mediante matrices ampliadas de  $3 \times 3$ . La matriz compuesta de traslación y rotación conjunta alrededor de un origen afín se define matemáticamente como:

$$M_{Transformacion} = \begin{bmatrix} \cos \theta & -\sin \theta & dx \\ \sin \theta & \cos \theta & dy \\ 0 & 0 & 1 \end{bmatrix}$$

Cada punto de la malla poligonal de la pieza se procesa multiplicando:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = M_{Transformacion} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

### B. Algoritmo de Envolvente Convexa - Graham Scan (Conceptos de Convexidad)

Para derivar la envolvente de la pieza irregular:

1. Se encuentra el punto con la menor coordenada Y (y menor X en caso de empate). Este punto se define como el pivote  $P_0$ .
2. El resto de los  $n_1$  puntos se ordenan en base al ángulo polar que forman con respecto a  $P_0$ .
3. Se recorren los puntos ordenados utilizando una estructura de datos de Pila (*Stack*). Para cada punto, se evalúa si el giro formado por los dos últimos elementos en la pila y el punto actual realiza un viraje hacia la izquierda (en sentido antihorario). Si el cálculo del determinante (producto cruz) es negativo o cero (giro a la derecha o colineal), significa que el punto intermedio rompe la convexidad estricta y se remueve de la pila (*pop*).

### C. Recorte de Polígonos - Sutherland-Hodgman (Unidad 1.4.6)

El polígono de la pieza se recorta de forma secuencial contra cada una de las cuatro rectas que definen las fronteras de la lámina (Izquierda, Derecha, Arriba, Abajo). Para cada plano de corte:

- Se evalúan los vértices del polígono por parejas consecutivas (aristas) formadas por un vértice de origen  $V_{in}$  y un vértice de destino  $V_{out}$ .
- Si ambos vértices están dentro de la frontera, el vértice de destino se añade a la lista de salida.

- Si el origen está dentro y el destino fuera, se calcula el punto exacto de intersección matemática de la recta de la arista con la recta del límite y se añade dicha intersección.
- Si el origen está fuera y el destino dentro, se calcula la intersección y se guarda tanto la intersección como el vértice de destino.

## 6.4 Pruebas

### Pruebas realizadas

- **Caso de Prueba 1 (Validación de Transformaciones Afines):** Se ingresó un polígono de prueba en forma de "L" descentrado. Se aplicó una rotación analítica pura de  $90^\circ$  y una traslación discreta. Se verificó manualmente mediante operaciones matriciales en papel que las coordenadas calculadas en el búfer interno del programa coincidieran exactamente con los vectores resultantes teóricos del espacio afín euclídeo.
- **Caso de Prueba 2 (Recorte en Fronteras Extremas):** Se posicionó intencionalmente una pieza rebasando la esquina superior derecha de la ventana limitadora de la lámina de corte. El algoritmo de *Sutherland-Hodgman* se ejecutó con éxito, dividiendo la arista comprometida y generando dos nuevos vértices coincidentes con las aristas ortogonales de la lámina. El cálculo del área útil mutó instantáneamente descartando la superficie externa.

### Resultados obtenidos y Validación

Las pruebas demostraron que los algoritmos gráficos tradicionales en 2D operan de manera perfectamente determinista. Al no existir hilos de renderizado continuo (*animation loops*), el procesador registró consumo de CPU de 0% durante los estados de reposo de la aplicación, incrementándose momentáneamente a fracciones mínimas únicamente en el instante preciso de recalcular el polígono ante interacciones discretas del usuario.

## 6.5 Implantación

### Forma de uso del sistema

1. **Fase de Carga:** El usuario define la geometría base haciendo clics discretos en la pantalla para sembrar los vértices que constituyen la pieza irregular inicial.
2. **Fase de Análisis de Convexidad:** Al presionar el comando estático "Calcular Envolvente", el sistema traza perimetralmente una línea continua perimetral que denota el *Convex Hull*.

3. **Fase de Ajuste y Acomodo Spatial:** El operario modifica los valores numéricos de traslación o rotación en la barra de comandos para ubicar estáticamente la pieza en distintas coordenadas dentro de la lámina.
4. **Fase de Análisis de Desperdicio:** La interfaz despliega los resultados matemáticos calculados de las áreas correspondientes.

**Requisitos técnicos:** Cualquier PC con Windows/ Linux o MacOS

## 7. Resultados

### Capturas del sistema e interfaces

Debido a la naturaleza estática del software, la salida del programa se compone de capturas de pantalla fijas de la sesión de optimización geométrica.

El lienzo presenta el siguiente esquema visual discretoizado por colores (**Modelos de Color - Unidad 2.4.1**):

- **Fondo del Sistema:** Gris claro mate para evitar fatiga visual en el operador.
- **Límites de la Lámina de Corte:** Un rectángulo estático negro de trazo grueso que define con precisión geométrica la ventana de recorte útil.
- **Área Interna de Piezas Válidas:** Polígonos rasterizados con relleno de color verde semitransparente, permitiendo verificar visualmente que las geometrías afines resultantes residen al 100% dentro del espacio de trabajo permitido.
- **Segmentos Excedentes o en Colisión:** En caso de que se sitúe la pieza fuera de los límites permisibles, el software intercepta los nodos excedentes y pinta las regiones truncadas por el recorte en color rojo sólido, denotando de forma clara al usuario que dicha porción ha sido removida matemáticamente por las líneas de corte del algoritmo de *Sutherland-Hodgman*.

Al pie del viewport, una tabla de texto estático despliega los valores numéricos calculados en tiempo real:

- Área Nominal de la Plancha: \$250,000 \text{ mm}^2\$
- Área Neta Ocupada por las Piezas: \$185,240 \text{ mm}^2\$
- Porcentaje de Aprovechamiento Neto: \$74.09\%\$

## 8. Discusión

### Ventajas de la solución

La principal ventaja de la herramienta diseñada radica en su absoluta estabilidad y predictibilidad matemática. Al apoyarse rigurosamente en las leyes de las transformaciones afines matriciales y las coordenadas homogéneas, no existen errores acumulativos de redondeo o desfase temporal que suelen plagar a las aplicaciones basadas en motores de física dinámica o hilos de animación basados en delta-time. Cumple de manera limpia y estricta el requerimiento pedagógico de resolver un problema computacional crítico mediante graficación pura y abstracta sin recurrir a simulaciones de movimiento en tiempo real.

### **Limitaciones**

La limitación de diseño más relevante es que el acomodo de las piezas sigue dependiendo de la interacción e inteligencia espacial discreta suministrada por el operador humano (el usuario debe cambiar manualmente los valores de rotación y traslación para buscar el óptimo). El sistema no cuenta en esta fase con un algoritmo de inteligencia artificial o búsqueda heurística automatizada para resolver el problema del *Nesting* de manera autónoma.

### **Problemas encontrados**

Durante la codificación del algoritmo de *Sutherland-Hodgman*, se detectó originalmente una anomalía en la que el polígono recortado se deformaba visualmente al salir por completo de uno de los extremos de la pantalla. Al examinar el pipeline matemático, se descubrió que cuando una pieza se sitúa totalmente en la región externa (ocluída), la lista de vértices de salida se vacía legítimamente, pero el pipeline de dibujo nativo intentaba trazar una primitiva lineal apuntando a referencias nulas. Se solucionó añadiendo una validación lógica condicional que detiene la rasterización de la pieza si el conteo de vértices devuelto por el clipping es menor a 3.

## **9. Conclusiones**

### **Cumplimiento del objetivo general**

Se dio cumplimiento integral al objetivo general trazado para el proyecto. Se estructuró un software utilitario funcional que mitiga y resuelve analíticamente el problema del desperdicio de material en procesos de corte industrial empleando herramientas fundamentales del temario de Graficación por Computadora. El proyecto demostró que la computación gráfica es una disciplina matemática profunda que trasciende por completo los límites del entretenimiento, los videojuegos o las animaciones fluidas,

constituyendo un pilar de optimización crítico en la ingeniería de software aplicada a la manufactura.

### **Aportes del proyecto**

El aporte principal de este desarrollo es el diseño e integración de una infraestructura matemática auditable que permite entender el pipeline de transformación y recorte de polígonos paso a paso. La metodología modular adoptada provee una base sólida de código limpio sobre la cual estudiantes o desarrolladores pueden comprender de manera transparente cómo interactúan las transformaciones en coordenadas homogéneas con los algoritmos clásicos de delimitación geométrica en un espacio euclídeo coordinado.

### **Relación con problemas reales o futuros trabajos**

Este software posee un enlace directo con problemas reales en la industria de la automatización y el control numérico (CNC). Como trabajos futuros, se plantea expandir los alcances del sistema mediante la integración de un módulo de optimización autónomo basado en Algoritmos Genéticos o colonias de hormigas que evalúe miles de combinaciones de matrices de transformación afín de manera automatizada para encontrar el acomodo geométrico óptimo sin intervención humana, así como la migración del pipeline geométrico hacia curvas paramétricas de Bézier o B-Splines para modelar contornos orgánicos complejos con un nivel superior de continuidad analítica.

### **10. Referencias (Formato IEEE)**

- [1] D. Hearn and M. P. Baker, *Computer Graphics with OpenGL*, 4th ed. Upper Saddle River, NJ: Prentice Hall, 2010.
- [2] J. D. Foley, A. van Dam, S. K. Feiner, and J. F. Hughes, *Computer Graphics: Principles and Practice*, 3rd ed. Boston, MA: Addison-Wesley, 2013.
- [3] R. L. Graham, "An Efficient Algorithm for Determining the Convex Hull of a Finite Planar Set," *Information Processing Letters*, vol. 1, no. 4, pp. 132-133, 1972.
- [4] I. E. Sutherland and G. W. Hodgman, "Reentrant Polygon Clipping," *Communications of the ACM*, vol. 17, no. 1, pp. 32-42, 1974.
- [5] F. P. Preparata and M. I. Shamos, *Computational Geometry: An Introduction*. New York, NY: Springer-Verlag, 1985.



## 11. Anexos

### Anexo A: Lógica matemática en Pseudocódigo del algoritmo Graham Scan (Convex Hull).

```
Función GrahamScan(Conjunto Puntos):  
    Encontrar punto  $P_0$  con menor coordenada Y (o menor X en empate)  
    Ordenar el resto de los Puntos por ángulo polar respecto a  $P_0$   
  
    Si tamaño de Puntos < 3 entonces retornar Puntos  
  
    Crear Pila S  
    S.push(Puntos[0])  
    S.push(Puntos[1])  
    S.push(Puntos[2])  
  
    Para i desde 3 hasta tamaño(Puntos) - 1 hacer:  
        Mientras S.tamaño() > 1 y ProductoCruzGiro(S.penúltimo(), S.último(), Puntos[i]) <= 0 hacer:  
            S.pop()  
            S.push(Puntos[i])  
  
    Retornar S (vértices de la envolvente convexa)
```

### Anexo B: Formulación del Producto Cruz para validación de Convexidad

Para determinar si tres puntos consecutivos  $A(x_1, y_1)$ ,  $B(x_2, y_2)$  y  $C(x_3, y_3)$  forman un giro a la izquierda (antihorario) o a la derecha (horario), el sistema evalúa el determinante de la matriz afín reducida, equivalente al componente Z del producto cruz de los vectores en el espacio euclídeo:

$$\text{Valor} = (x_2 - x_1)(y_3 - y_1) - (y_2 - y_1)(x_3 - x_1)$$

- Si Valor > 0: El giro es hacia la izquierda (Vértice convexo válido).
- Si Valor <= 0: El giro es a la derecha o colineal (No convexo, el vértice intermedio debe ser descartado del *Convex Hull*).